# XO Arena — Bot Training Guide

A practical reference for creating and training competitive bots. Each algorithm has different strengths, convergence speed, and ideal session structure. Follow the recipes below for the best results.

---

## Table of Contents


1. Core Concepts
2. Algorithm Overview
3. Q-Learning
4. SARSA
5. Monte Carlo
6. Policy Gradient
7. DQN
8. AlphaZero
9. Benchmarking & Evaluation
10. Troubleshooting


---

## Core Concepts

### Epsilon (ε) — Exploration vs. Exploitation

Epsilon controls the probability of choosing a **random move** instead of the agent's current best move.

- **ε = 1.0** — fully random (pure exploration, learns nothing yet)
- **ε = 0.5** — 50/50 mix
- **ε = 0.05** — mostly greedy, small residual exploration
- **ε = 0.0** — fully greedy (exploitation only, no learning)

Epsilon starts high and decays over time. Decaying too fast locks in a bad policy before it has converged. Decaying too slow wastes episodes on random play once the agent already knows the right moves.

### Epsilon Decay Methods

| Method | Formula | Best For |
|---|---|---|
| **Exponential** | `ε *= decay_rate` each episode | Most algorithms (default) |
| **Linear** | ε decreases evenly over N episodes | DQN, fixed budget |
| **Cosine** | Cosine curve from start → min | Smoother warmup/cooldown |

**Rule of thumb:** ε should reach `epsilonMin` at roughly **80% of total planned episodes**, leaving the last 20% for pure exploitation to lock in the learned policy.

### Epsilon Decay Rate Quick Reference

> *In the app, this is the **"Rate"** field (visible when "Decay schedule" = Exponential).*

| Rate | Episodes to reach ε=0.05 (starting at ε=1.0) |
|---|---|
| 0.995 | ~590 |

| | |
|---|---|
| 0.999 | ~2,990 |
| 0.9995 | ~5,990 |
| 0.9999 | ~29,950 |
| 0.99995 | ~59,900 |

### Training Modes

| Mode | Description | Best For |
|---|---|---|
| **Self-play** | Bot plays both sides simultaneously | All algorithms, best for tabular methods |
| **vs Minimax** | Bot plays against a deterministic opponent | Curriculum training, final polish |
| **Alternating** | Bot plays X one episode, O the next | Ensures symmetry for both sides |

### Curriculum Training

Available in vs-Minimax mode. The system automatically advances difficulty (novice → intermediate → advanced → master) once the bot wins 65% of the last 100 games. Start at **novice** — jumping straight to advanced wastes early episodes against an opponent the agent cannot yet learn from.

### Gamma (γ) — Discount Factor

Controls how much future rewards are valued vs immediate ones.

- **γ = 0.9** — standard, good for tic-tac-toe (short games)
- **γ = 0.99** — near-sighted future; used for AlphaZero
- Lower γ → agent plays for quick wins; higher γ → agent plans further ahead

For tic-tac-toe specifically, γ = 0.9 is ideal since games are at most 9 moves.

> ***In the app:*** *γ is exposed in the* ***DQN Train tab*** *as the* ***"Gamma (γ)"*** *dropdown (0.85 / 0.90 / 0.95 / 0.99). For all other algorithms it is hardcoded (0.9 for tabular, 0.99 for AlphaZero). For tabular methods only, γ can also be swept via the* ***Auto-Tuner*** *tab.*

### Learning Rate (α)

Controls how aggressively Q-values are updated per step.

- **Tabular methods (Q-Learning, SARSA, MC):** α = 0.3–0.5 is safe. Higher is faster but noisier.
- **Neural net methods (DQN, AlphaZero):** α = 0.001. Too high causes divergence.

> ***In the app:*** *α is* ***not exposed in the Train tab*** *for any algorithm. It is hardcoded at 0.3 (tabular) and 0.001 (neural net). For tabular methods only, α can be swept via the* ***Auto-Tuner*** *tab.*

↑ Back to top

---

## Algorithm Overview

| Algorithm | Type | State Space | Episodes to Competent | Best Strength |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| **Q-Learning** | Tabular, off-policy TD | Explicit table | 3,000–8,000 | Fast, reliable, debuggable |
| **SARSA** | Tabular, on-policy TD | Explicit table | 4,000–10,000 | Conservative, safe policy |
| **Monte Carlo** | Tabular, episode-level | Explicit table | 5,000–15,000 | No bootstrapping bias |
| **Policy Gradient** | Tabular softmax | Explicit table | 8,000–20,000 | Stochastic, hard to exploit |
| **DQN** | Neural net, off-policy | Neural network | 10,000–25,000 | Generalizes unseen states |
| **AlphaZero** | Neural net + MCTS | Neural network | 5,000–15,000 | Strongest ceiling |

**If you want the strongest bot:** AlphaZero > DQN > Q-Learning. **If you want the fastest to train:** Q-Learning > SARSA > Monte Carlo.

↑ Back to top

---

## Q-Learning

**Algorithm:** Off-policy TD control. Updates Q(s,a) immediately after each move using the best possible next action (regardless of what will actually be played).

**State space:** Explicit table keyed by board string. Tic-tac-toe has ~5,478 reachable states — Q-Learning covers this completely within a few thousand episodes.

### Recommended Settings

| UI Label | Value | Configurable? | Notes |
|---|---|---|---|
| Decay schedule | **Exponential** | Train tab | |
| Rate | **0.995** | Train tab (Exponential only) | Reaches ε≈0.05 by ~590 episodes |
| Epsilon min | **0.05** | Train tab | 5% residual exploration |
| Reset ε to 1.0 at start | **checked** | Train tab | Always start fully random |
| Learning rate (α) | 0.3 | **Hardcoded** | Not shown in UI — Auto-Tuner can sweep it |
| Discount factor (γ) | 0.9 | **Hardcoded** | Not shown in UI — Auto-Tuner can sweep it |

### Session Recipe

| Session | Episodes | Mode | Opponent | Purpose |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| 1 | 2,000 | Self-play | — | Bootstrap all states |
| 2 | 3,000 | Self-play | — | Converge Q-table |
| 3 | 2,000 | vs Minimax | Curriculum (novice→master) | Polish vs deterministic play |

**Total: ~7,000 episodes**

### Expected Results by Episode Count

| Episodes | vs Random | vs Easy Minimax | vs Hard Minimax |
|---|---|---|---|
| 500 | 50–60% | 20–30% | <5% |
| 2,000 | 75–85% | 50–65% | 10–20% |
| 5,000 | 85–92% | 70–80% | 30–50% |
| 8,000+ | 90–95% | 80–90% | 50–65% |

### Tips

- Q-Learning is **off-policy**: it learns the greedy strategy even while exploring. This makes it faster than SARSA at reaching a strong policy.
- Self-play is ideal for session 1 because it trains both X and O simultaneously.
- Once the bot reaches ~80% vs random, switch to curriculum minimax to sharpen strategy.
- If the Q-table stalls (win rate plateaus for 2,000+ episodes), lower α to 0.1 and run 2,000 more episodes — aggressive learning rates overwrite good values at low epsilon.

↑ Back to top

---

## SARSA

**Algorithm:** On-policy TD control. Updates Q(s,a) using the action that will *actually* be taken next (not the greedy max). This makes SARSA more conservative — it learns a safer policy that accounts for its own exploration behavior.

**State space:** Same explicit table as Q-Learning.

### Recommended Settings

| UI Label | Value | Configurable? | Notes |
|---|---|---|---|
| Decay schedule | **Exponential** | Train tab | |
| Rate | **0.995** | Train tab (Exponential only) | |
| Epsilon min | **0.05** | Train tab | Keep slightly higher than Q-Learning |
| Reset ε to 1.0 at start | **checked** | Train tab | |

| | | | |
|---|---|---|---|
| Learning rate (α) | 0.3 | **Hardcoded** | Not shown in UI — Auto-Tuner can sweep it |
| Discount factor (γ) | 0.9 | **Hardcoded** | Not shown in UI — Auto-Tuner can sweep it |

### Session Recipe

| Session | Episodes | Mode | Opponent | Purpose |
|---|---|---|---|---|
| 1 | 3,000 | Self-play | — | Explore all states under SARSA's cautious policy |
| 2 | 3,000 | Self-play | — | Converge |
| 3 | 2,000 | vs Minimax | Curriculum | Polish |

**Total: ~8,000 episodes**

### Expected Results

| Episodes | vs Random | vs Easy | vs Hard |
|---|---|---|---|
| 1,000 | 45–55% | 15–25% | <5% |
| 4,000 | 70–80% | 55–65% | 15–25% |
| 8,000+ | 85–90% | 70–80% | 40–55% |

### SARSA vs Q-Learning

- SARSA converges **~20% slower** but produces a policy that is less exploitable in practice.
- The gap between the two narrows as epsilon decays to 0 — at inference time (ε=0) both become identical greedy agents.
- Prefer SARSA if you want a bot that draws rather than gambles; prefer Q-Learning if you want raw win rate.

↑ Back to top

---

## Monte Carlo

**Algorithm:** Every-visit MC control. Waits until the end of the episode, then propagates the actual discounted return backward through all visited (state, action) pairs. No bootstrapping — every update is based on real experienced outcomes.

**Convergence:** Slower than TD methods because updates only happen at episode end, not after each move. However, updates are unbiased — there is no bootstrapping error accumulating over time.

### Recommended Settings

| UI Label | Value | Configurable? | Notes |
|---|---|---|---|

| | | | |
|---|---|---|---|
| Decay schedule | **Linear** | Train tab | Linear works well for MC; more even coverage |
| Epsilon min | **0.05** | Train tab | |
| Reset ε to 1.0 at start | **checked** | Train tab | |
| Learning rate (α) | 0.2 | **Hardcoded** | Not shown in UI — lower than TD because MC updates are noisier; Auto-Tuner can sweep it |
| Discount factor (γ) | 0.9 | **Hardcoded** | Not shown in UI — Auto-Tuner can sweep it |

> ***Note:*** *Monte Carlo does not show a "Rate" field when Linear decay is selected — the decay spreads evenly across the session's episode count automatically.*

### Session Recipe

| Session | Episodes | Mode | Opponent | Purpose |
|---|---|---|---|---|
| 1 | 5,000 | Self-play | — | Wide exploration — many unique trajectories |
| 2 | 5,000 | Self-play | — | Reinforce good trajectories |
| 3 | 3,000 | vs Minimax | Curriculum | Tighten strategy |

**Total: ~13,000 episodes**

### Expected Results

| Episodes | vs Random | vs Easy | vs Hard |
|---|---|---|---|
| 2,000 | 40–55% | 15–25% | <5% |
| 6,000 | 65–75% | 45–60% | 10–20% |
| 12,000+ | 80–88% | 65–75% | 30–45% |

### Tips

- Use **linear decay** — Monte Carlo benefits from staying exploratory longer because each episode covers a unique trajectory. Exponential decay collapses exploration too early.
- MC is the most data-hungry algorithm but has the cleanest credit assignment: every move in a winning game is credited, every move in a losing game is penalized.
- The final policy quality is often comparable to Q-Learning at 1.5–2x the episode count.

↑ Back to top

---

## Policy Gradient

**Algorithm:** REINFORCE with a tabular softmax policy. Instead of learning Q-values and acting greedy, it directly learns action preferences θ(s,a) and samples from a probability distribution. This produces naturally stochastic play — the bot doesn't always make the same move in the same position.

**Key difference from Q-Learning:** The policy never fully "locks in" a deterministic move. At inference, it picks the highest-preference action, but the learned distribution means it can be less predictable against human opponents.

### Recommended Settings

| UI Label | Value | Configurable? | Notes |
|---|---|---|---|
| Decay schedule | **Cosine** | Train tab | Smooth schedule suits PG's gradient updates |
| Epsilon min | **0.05** | Train tab | |
| Reset ε to 1.0 at start | **checked** | Train tab | Controls fallback to random; PG uses softmax sampling internally |
| Learning rate (α) | 0.01 | **Hardcoded** | Not shown in UI — PG is sensitive; hardcoded lower than tabular default; Auto-Tuner can sweep it |
| Discount factor (γ) | 0.9 | **Hardcoded** | Not shown in UI — Auto-Tuner can sweep it |

> ***Note:*** *Policy Gradient does not show a "Rate" field when Cosine decay is selected.*

### Session Recipe

| Session | Episodes | Mode | Opponent | Purpose |
|---|---|---|---|---|
| 1 | 5,000 | Self-play | — | Build preference distribution across states |
| 2 | 5,000 | Self-play | — | Reinforce high-return trajectories |
| 3 | 5,000 | vs Minimax | Curriculum | Sharpen vs deterministic opponent |
| 4 | 3,000 | vs Minimax | Master | Final polish |

**Total: ~18,000 episodes**

### Expected Results

| Episodes | vs Random | vs Easy | vs Hard |
|---|---|---|---|
| 2,000 | 35–50% | 10–20% | <5% |
| 8,000 | 60–72% | 40–55% | 10–20% |
| 15,000+ | 75–85% | 60–72% | 25–40% |

### Tips

- Policy Gradient is the **slowest tabular algorithm** but produces the most unpredictable play.
- The learning rate is critical: α > 0.05 causes oscillation. If the win rate bounces wildly between sessions, halve α.
- The cosine decay schedule works particularly well here — it mirrors the natural "explore then refine" lifecycle of REINFORCE.

- PG bots make better opponents for human players because they are not fully deterministic.

↑ Back to top

---

## DQN (Deep Q-Network)

**Algorithm:** Neural network function approximation for Q-values. Instead of an explicit state table, a small MLP `[9 → hidden → 9]` represents Q-values. Uses experience replay (circular buffer) and a separate target network for stability.

**Key advantages:**

- Can generalize across similar board states that were never exactly seen during training.
- No state table — model size stays fixed regardless of states visited.
- With Adam optimizer and the corrected adversarial Bellman equation, converges reliably.

**DQN Train tab controls:**

| UI Label | Default | Description | Configurable? |
|---|---|---|---|
| **Batch** | 32 | Samples per gradient step | Yes — Train tab |
| **Replay buffer** | 10,000 | Experiences stored for random sampling | Yes — Train tab |
| **Target update** | 100 | Steps between target network syncs | Yes — Train tab |
| **Network architecture** | [32] | 1–3 hidden layers, each 8/16/32/64/128 neurons | Yes — Train tab |
| **Gamma (γ)** | 0.90 | Discount factor (0.85 / 0.90 / 0.95 / 0.99) | Yes — Train tab |
| Decay schedule | Exponential | Epsilon decay curve | Yes — Train tab |
| Rate | 0.9999 | Per-episode decay multiplier (Exponential only) | Yes — Train tab |
| Epsilon min | 0.05 | Minimum epsilon floor | Yes — Train tab |
| Reset ε to 1.0 at start | checked | Whether to restart exploration | Yes — Train tab |
| Learning rate (α) | 0.001 | Adam optimizer step size | **Hardcoded** — not exposed |

> ***Architecture changes reset weights.*** *If you change the layer layout (e.g., [32] → [64, 64]) the Train tab will show an amber warning and the existing trained weights will be discarded — training starts fresh with the new shape. The model's stored architecture is updated to match after the session completes.*

### Recommended Settings

| UI Label | Value | Notes |
|---|---|---|

| | | |
|---|---|---|
| Network architecture | **[64, 64]** | Two layers; best quality/speed tradeoff for tic-tac-toe |
| Gamma (γ) | **0.95** | Better than default 0.90 — plans further ahead in 9-move games |
| Decay schedule | **Exponential** | Required for multi-session training — see note below |
| Rate | **0.9999** | Reaches ε≈0.05 by ~29,950 episodes across all sessions |
| Epsilon min | **0.05** | |
| Reset ε to 1.0 at start | **checked** | |
| Batch | **32** | Default; increase to 64 for [128, 128] nets |
| Replay buffer | **10,000** | Good for 30k-episode runs |
| Target update | **100** | Default; do not lower below 50 |

> ***Why Exponential for DQN?*** *Linear and Cosine decay spread ε from its current value to Epsilon min evenly across one session's episodes — the Rate field is hidden because it isn't used. For a 10k-episode session starting at ε=1.0 with Epsilon min=0.05, Linear would reach the floor at episode 10,000, leaving sessions 2–4 with no exploration. Exponential at Rate 0.9999 carries exploration smoothly across 30,000+ episodes, matching the ε estimates in the session recipe below. Use Linear/Cosine only for a single all-in-one session.*

### Session Recipe

The **ε start → end** column shows predicted values based on Exponential 0.9999 decay — these are not inputs you configure. ε start is determined by the previous session's stored end state. The only epsilon control in the Train tab is the **"Reset ε to 1.0"** checkbox:

- **Session 1** — check it (start fresh at 1.0)
- **Sessions 2–4** — uncheck it (continue from the stored value left by the previous session)

ε end is likewise not settable; it is simply where epsilon lands after N episodes of decay at your chosen Rate.

| Session | Episodes | Mode | Opponent | ε start → end (predicted) | Reset ε to 1.0 | Purpose |
|---|---|---|---|---|---|---|
| 1 | 10,000 | Self-play | — | 1.0 → ~0.37 | **checked** | Warm up replay buffer, learn basic move validity |
| 2 | 10,000 | Self-play | — | ~0.37 → ~0.14 | unchecked | Core policy learning |

| | | | | | | |
|---|---|---|---|---|---|---|
| 3 | 10,000 | vs Minimax | Curriculum (novice→advanced) | ~0.14 → ~0.05 | unchecked | Strategic refinement |
| 4 | 5,000 | vs Minimax | Master | ~0.05 → ~0.05 | unchecked | Final hardening |

**Total: ~35,000 episodes**

### Expected Results

| Episodes | vs Random | vs Easy | vs Hard |
|---|---|---|---|
| 5,000 | 50–65% | 20–35% | <10% |
| 15,000 | 72–82% | 50–65% | 20–35% |
| 25,000 | 82–90% | 65–78% | 35–50% |
| 35,000+ | 88–93% | 75–85% | 50–65% |

### Understanding the Replay Buffer

The replay buffer is a **circular memory** that stores past game experiences as `(board_state, action, reward, next_board_state, done)` tuples. During each training step the DQN samples a random mini-batch from this buffer — it does *not* train on the episode it just played.

**Why this matters:**

- **Breaks temporal correlation** — consecutive episodes are highly correlated (same opening, similar board patterns). Random mini-batch sampling produces more diverse batches, which stabilises gradient descent and prevents catastrophic forgetting.
- **Reuses rare experiences** — important board positions (e.g. fork threats, game-ending states) can be revisited many times even if they are infrequently encountered. Without the buffer each experience is used exactly once and then discarded.

**Buffer size relative to episode count:**

| Episodes trained | Buffer = 10,000 | Buffer = 20,000 |
|---|---|---|
| 5,000 | Buffer is half full; all experiences available | Buffer is quarter full |
| 10,000 | Buffer exactly full; oldest evicted as new arrive | Still holds all 10k experiences |
| 30,000 | Only the most recent 10k episodes are kept | Keeps the most recent 20k |

A buffer that is **too small** for long runs means the network only learns from recent play — it can forget earlier lessons and oscillate. Increase to 20,000–50,000 for sessions over 20,000 episodes.

A buffer that is **too large** simply has empty slots early in training; this is harmless — the network trains on what is available.

**When to tune the replay buffer:**

- Win rate oscillates wildly → buffer too small (increase it)
- Bot forgets good moves it knew earlier → buffer too small relative to total episodes trained

- Training speed feels slow on a powerful machine → batch size is the knob to increase, not buffer size

### Understanding the Training Charts

The live training panel shows two sets of lines for win/draw/loss rates:

- **Thick solid lines** — the *recent* rate for the current batch of episodes only (resets each update interval)
- **Thin dashed lines** — the *cumulative* average since the session started

The recent lines are the most informative signal for whether the model is actively learning. If a bot is genuinely improving — for example, drawing more in the last 100 episodes than it did in the first 1,000 — the recent draw rate line will climb while the cumulative line rises slowly (weighted down by early losses).

**If both lines are flat**, the model is not converging in this session.

### Network Architecture Guide

Network architecture is now configurable **in the Train tab** using the layer builder. You can add 1–3 hidden layers, each with 8, 16, 32, 64, or 128 neurons. Changing the architecture resets the model's weights — the amber warning in the UI will tell you when this will happen.

| Architecture | Parameters | Training Speed | Recommended For |
|---|---|---|---|
| `[32]` | ~380 | Fastest | Quick experiments |
| `[64]` | ~700 | Fast | Good single-layer baseline |
| `[64, 64]` | ~1,350 | Medium | **Best quality/speed tradeoff** |
| `[128, 64]` | ~2,700 | Slower | Diminishing returns for tic-tac-toe |

For tic-tac-toe, `[64, 64]` is the sweet spot — larger networks don't raise the ceiling but take longer to train.

### Tips

- Always use **self-play for sessions 1–2**. The replay buffer fills faster and both X and O perspectives are covered simultaneously.
- Use **Exponential decay with Rate 0.9999**. Linear/Cosine decay from ε=1.0 to 0.05 within a single 10k-episode session — leaving nothing for future sessions. Exponential at 0.9999 spreads exploration gradually across all sessions.
- The minimum viable run is **15,000 episodes** to fill and recycle the replay buffer enough for the Bellman targets to stabilize.
- If the win rate is flat after 20,000 episodes: change the architecture to `[64, 64]` in the Train tab (weights reset, but the new capacity helps more than extra episodes on a small net).
- Use **Gamma = 0.95** — it lets the agent plan further ahead without introducing instability in 9-move games.

↑ Back to top

---

## AlphaZero

**Algorithm:** Monte Carlo Tree Search (MCTS) guided by two neural networks — a policy net `[9→64→32→9]` (softmax output) and a value net `[9→64→32→1]` (tanh output). Each episode runs `numSimulations` MCTS rollouts to build a visit-count distribution, then trains both networks on the collected examples.

**Key advantages:**

- MCTS guarantees the agent considers multiple lookahead paths before each move — not just immediate reward.
- The value network provides a learned evaluation function.
- No epsilon — exploration is naturally built into the PUCT tree search.
- Consistently produces the strongest policy of all algorithms given sufficient episodes.

**AlphaZero Train tab controls:**

| UI Label | Default | Description | Configurable? |
|---|---|---|---|
| **Simulations** | 50 | MCTS rollouts per move. Higher = stronger but slower. | Yes — Train tab |
| **PUCT** | 1.5 | Exploration constant in tree search. Higher = more exploration. | Yes — Train tab |
| **Temperature** | 1.0 | Randomness in move selection from visit counts. | Yes — Train tab |
| Learning rate (α) | 0.001 | Shared learning rate for policy + value nets | **Hardcoded** |
| Discount factor (γ) | 0.99 | Future reward weighting | **Hardcoded** |

> *AlphaZero has **no epsilon** — exploration is naturally built into the PUCT tree search. There is no "Exploration" section for AlphaZero in the Train tab.*

### Recommended Settings

| UI Label | Value | Configurable? | Notes |
|---|---|---|---|
| Simulations | **100** | Train tab | Double the default for much stronger search |
| PUCT | **1.5** | Train tab | Default is good; increase to 2.0 for more exploration early |
| Temperature | **1.0** | Train tab | Reduce to 0.5 after first 5,000 episodes for more decisive play |
| Learning rate (α) | 0.001 | **Hardcoded** | Adam optimizer; not exposed in any tab |
| Discount factor (γ) | 0.99 | **Hardcoded** | Not exposed; higher than tabular default because AZ plans further ahead |
| Epsilon / exploration | — | **Not applicable** | AlphaZero has no epsilon — PUCT handles all exploration in the tree |

### Session Recipe

| Session | Episodes | Simulations | Temperature | Purpose |
|---|---|---|---|---|
| 1 | 3,000 | 50 | 1.0 | Fast bootstrap — build initial policy/value estimates |
| 2 | 5,000 | 100 | 1.0 | Deeper search, refine both nets |
| 3 | 5,000 | 100 | 0.5 | More decisive play, sharpen value net |
| 4 | 2,000 | 200 | 0.1 | Near-deterministic fine-tuning |

**Total: ~15,000 episodes**

> ***Note:*** *AlphaZero episodes are significantly slower than tabular episodes because each move runs* `numSimulations` *MCTS rollouts. Expect 5–10x more wall-clock time per episode vs Q-Learning.*

### Expected Results

| Episodes | vs Random | vs Easy | vs Hard |
|---|---|---|---|
| 1,000 | 60–72% | 35–50% | 10–20% |
| 5,000 | 78–87% | 60–75% | 35–50% |
| 10,000 | 87–93% | 75–85% | 55–70% |
| 15,000+ | 92–97% | 82–90% | 65–80% |

AlphaZero reaches competency **much faster per-episode** than other algorithms because MCTS provides strong implicit lookahead even early in training. The episode count is lower but wall-clock time is higher.

### Tips

- **Self-play only** — AlphaZero is designed exclusively for self-play. It has no concept of an external opponent function.
- Start with **Simulations=50** to fill the experience buffer quickly, then increase to 100+ as the networks gain accuracy.
- **Temperature** controls how the final move is sampled from MCTS visit counts. At 1.0, proportional sampling adds diversity. At 0.1, the most-visited child is almost always chosen. Reduce Temperature over training for a sharper policy.
- **PUCT=1.5** balances exploitation vs exploration in the tree. If the bot gets stuck in repetitive patterns early, try PUCT=2.0.
- AlphaZero's policy net outputs are probabilities, not Q-values. The Explainability tab shows these as move probabilities — a strong AZ bot will show clear high-probability cells for center and corner openings.

↑ Back to top

---

## Benchmarking & Evaluation

Always run a benchmark after training to get objective scores before deploying a bot.

### What the Benchmark Measures

| Scenario | Opponent | Win Rate Goal (good bot) |
|---|---|---|
| vs Random | Pure random | > 85% |
| vs Easy | Novice minimax | > 70% |
| vs Medium | Intermediate minimax | > 55% |
| vs Tough | Advanced minimax | > 35% |
| vs Hard | Perfect minimax | > 15% (draws are good here) |

> *A perfect tic-tac-toe player can never lose — only win or draw. "vs Hard" scores above 15% indicate the bot is finding genuine wins against a perfect opponent by exploiting first-move advantage.*

### Benchmark-to-ELO Correlation

| Avg Benchmark Win Rate | Approximate ELO |
|---|---|
| < 30% | < 900 |
| 30–45% | 900–1,050 |
| 45–60% | 1,050–1,200 |
| 60–72% | 1,200–1,400 |
| 72–82% | 1,400–1,600 |
| > 82% | 1,600+ |

### Head-to-Head: When to Use

Use head-to-head (H2H) to compare two candidate models trained with different configs. Run at least **200 games** for statistical significance. The ELO update from H2H reflects the true relative strength better than benchmark scores against fixed opponents.

### p-Value Interpretation

Every benchmark result includes a p-value testing whether win rate > 50%.

- **p < 0.05** — the result is statistically significant (bot is genuinely performing above chance)
- **p > 0.10** — sample size too small or the bot is near 50% — run more games or more training

↑ Back to top

---

## Troubleshooting

### Win rate is flat / not improving

1. **Rate (epsilon decay) too fast** — check what epsilon is at the stall point. If ε < 0.1 and the bot hasn't converged, restart with a slower Rate (e.g. 0.999 → 0.9999).
2. **Learning rate too high** — for tabular methods: use the Auto-Tuner to find a better α. For DQN: α is hardcoded at 0.001 (well-tuned, not the issue).

3. **Opponent mismatch** — don't train vs Hard minimax until the bot reliably beats Easy. Use curriculum.
4. **Too few episodes before vs-minimax** — always do self-play first to bootstrap all states/weights before switching to a fixed opponent.

### Win rate is oscillating wildly

1. **Learning rate too high** — for tabular methods: use the Auto-Tuner to sweep α. For DQN: α is fixed at 0.001 (this isn't the cause).
2. **DQN only: Replay buffer too small** — if "Replay buffer" < 5,000 and you're doing 10k+ episodes, old experiences are evicted before they can be replayed enough. Increase to 20,000.
3. **DQN only: Target update too low** — syncing the target network every 10 steps instead of 100 causes instability. Keep "Target update" at 100+.

### Q-Learning / SARSA / MC win rate hits 80% then never improves

The tabular Q-table is fully converged. Switching algorithms (to DQN or AlphaZero) is the right move — not more episodes of the same algorithm.

### AlphaZero is very slow

Each episode runs "Simulations" MCTS passes, each a full tree traversal. At Simulations=200 this is ~200× slower than a Q-Learning episode. Start with 50, confirm the bot is learning (rising win rate on the analytics tab), then increase.

### Bot plays well as X but poorly as O (or vice versa)

The bot was trained asymmetrically. Run 2,000–3,000 additional self-play episodes using **alternating** mode so both X and O perspectives receive balanced updates.

### DQN wins a lot during training but benchmarks poorly

Training win rate is measured with exploration active (ε > 0). Benchmark uses pure exploitation (ε = 0). If the bot's greedy policy is weak, more training with a lower epsilon floor is needed. Set **"Epsilon min" to 0.01** and run 5,000 more episodes.

↑ Back to top